

Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology



Department of Mechatronics Engineering

Course No.: MTE 3104

Course Title: Microcontroller & Interfacing Sessional

Experiment No.: 09

Experiment Name: Design & Implementation of Password Based Door Locking System Using AVR ATmega32 LCD Display, Matrix Keypad, and Servo Motor

<u>Remarks</u>

<u>Submitted By:</u>	<u>Submitted To:</u>
Bayazid Habib Siddiquee 3 rd year (Odd Semester) Roll:2208053 Department of Mechatronics Engineering, RUET	Prangon Das Assistant Professor, Department of Mechatronics Engineering, RUET

Date of Experiment:

Date of Submission: 5/5/2026

Experiment No.: 09

Experiment Name: Design & Implementation of Password Based Door Locking System Using AVR ATmega32 LCD Display, Matrix Keypad, and Servo Motor

Objective

- To design a **password-based security system** using ATmega32.
- To interface a **4×4 matrix keypad** for password input.
- To display system status on a **16×2 LCD**.
- To control a **servo motor** for door locking/unlocking.
- To enhance understanding of embedded security systems

Required Equipment's:

- Arduino UNO, ATmega32
- LCD Display, 1k ohm resistance, jumpers
- Keypad
- Tools: avr-gcc, avr-objcopy, avr-dude or Arduino IDE
- Servo motor, buzzer

Introduction:

Embedded systems have become a cornerstone of modern electronics, enabling intelligent control and automation in everyday devices. The ATmega32 microcontroller, a member of Atmel's popular AVR family, is one of the most widely adopted 8-bit microcontrollers in educational and professional embedded system design. With its 32 KB of flash memory, built-in analog-to-digital converter, hardware timers, and support for serial communication protocols such as SPI and USART, the ATmega32 provides a compact yet capable platform for a wide range of applications. [1]

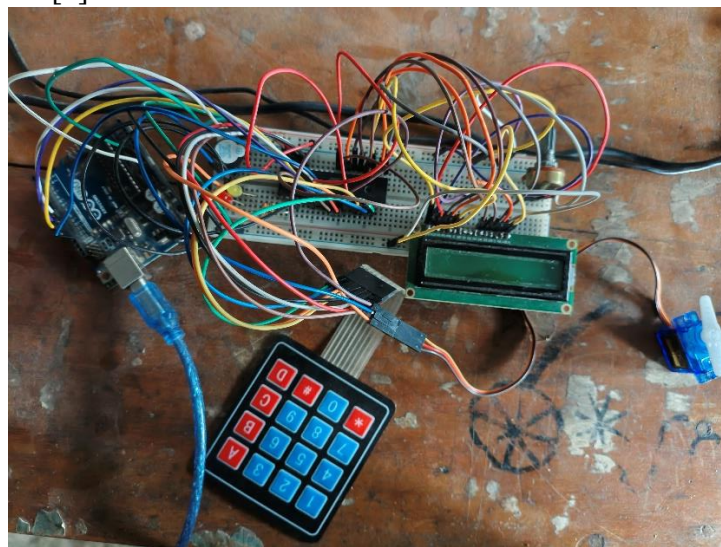


Fig 1: Controlling LCD with the help of 4x4 Keypad Matrix

A **password-based door locking system** is an electronic security system where access is granted only after entering a correct password. The **keypad** is used to enter the password. The **ATmega32 microcontroller** processes the input and compares it with a predefined password. The **LCD display** shows messages such as *Enter Password*, *Access Granted*, or *Access Denied*. A **servo motor** acts as the locking mechanism. When the correct password is entered, the servo

rotates to unlock the door; otherwise, it remains locked. Servo motors are controlled using **PWM (Pulse Width Modulation)** signals. The angle of rotation depends on the pulse width.

Procedure:

Step 1: Connecting Arduino UNO with Atmega32

Signal	Arduino Uno pin	ATmega8 pin	ATmega32 pin	Description
MOSI	D11 (PB3)	Pin 17 PB3/MOSI	Pin 6 PB5/MOSI	Master out — data to target
MISO	D12 (PB4)	Pin 18 PB4/MISO	Pin 7 PB6/MISO	Master in — data from target
SCK	D13 (PB5)	Pin 19 PB5/SCK	Pin 8 PB7/SCK	SPI clock
RESET	D10 (SS)	Pin 1 PC6/RESET	Pin 9 RESET	Puts target into ISP mode
VCC	5 V	Pins 7 & 20 VCC/AVCC	Pins 10 & 30 VCC/AVCC	Supply voltage (5 V)
GND	GND	Pins 8 & 22 GND/AGND	Pins 11 & 31 GND/AGND	Ground — connect all GND pins

Table 1: Pin diagram of ATmega8, ATmega32 and UNO to Transfer Data Among Them

Step 2: There is a 10 micro farad capacitor in between RESET and GND pin so ATmega8 can be reset and upload new files safely. And the pin diagram of my project is:

LCD Display: PORTC with D0 – D7	Keypad 4x4: Rows: PD0, PD1, PD2, PD3 Cols: PD4, PD5, PD6, PB1
---	--

Table 2: Pin diagram

Step 3: As I'm linux user I needed to install avr-gcc to compile C code of ATmega8, avr-objcopy to make a runnable script on ATmega8 and last avr-dude to upload and burn ATmega8. All the script I have built as [run.sh](#) and the ATmega code as [main.c](#) and for lcd I built [lcd.h](#).

Step 4: Run the script and it uploaded the main.c in ATmega8 and LED was on. And was able to control using IR sensor and keypad

Code:

[run.sh](#):

```
avr-gcc -mmcu=atmega8 -DF_CPU=1000000UL -Os -o counter.elf main.c
avr-objcopy -O ihex -R .eeprom counter.elf counter.hex
avrdude -c stk500v1 -p m8 -P /dev/ttyACM0 -b 19200 -B 250 -U
flash:w:counter.hex:i
```

[main.c](#):

```
#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <string.h>
#include "lcd.h"

// Keypad
#define ROW_PORT PORTA
#define ROW_DDR DDRA
```

```

#define ROW_PIN PINA

#define COL_PORT PORTB
#define COL_DDR DDRB
#define COL_PIN PINB

char keypad[4][4] = {
    {'1','2','3','A'},
    {'4','5','6','B'},
    {'7','8','9','C'},
    {'*','0','#','D'}
};

char password[] = "1234";
char entered[5];

// Servo (OC1A -> PD5)
void servo_init() {
    DDRD |= (1 << PD5);

    // Fast PWM, ICR1 as TOP
    TCCR1A = (1<<COM1A1) | (1<<WGM11);
    TCCR1B = (1<<WGM12) | (1<<WGM13) | (1<<CS11); // prescaler 8

    ICR1 = 19999; // 20ms period (50Hz)
}

// Rotate servo
void servo_angle(uint16_t angle) {
    OCR1A = 1000 + ((angle * 1000) / 180); // approx mapping
}

// Keypad init
void keypad_init() {
    ROW_DDR = 0x0F;
    ROW_PORT = 0xF0;

    COL_DDR = 0x00;
    COL_PORT = 0xFF;
}

// Get key
char keypad_getkey() {
    for (int row = 0; row < 4; row++) {
        ROW_PORT = ~(1 << row);

        for (int col = 0; col < 4; col++) {
            if (!(COL_PIN & (1 << col))) {
                _delay_ms(50);
                while (!(COL_PIN & (1 << col)));
                return keypad[row][col];
            }
        }
    }
}

```

```

    return 0;
}

int main(void) {
    lcd_init();
    keypad_init();
    servo_init();

    servo_angle(0); // Locked position

    while (1) {
        lcd_clear();
        lcd_string("Enter Password:");

        for (int i = 0; i < 4; i++) {
            char key = 0;
            while (!key)
                key = keypad_getkey();

            entered[i] = key;
            lcd_data('*'); // hide password
        }
        entered[4] = '\0';

        if (strcmp(entered, password) == 0) {
            lcd_clear();
            lcd_string("Access Granted");

            servo_angle(90); // Unlock
            _delay_ms(3000);

            servo_angle(0); // Lock again
        } else {
            lcd_clear();
            lcd_string("Access Denied");
            _delay_ms(2000);
        }
    }
}

```

Lcd.h

```

void lcd_command(unsigned char cmd){
    LCD_PORT = (LCD_PORT & 0x87) | (cmd & 0xF0); // High nibble
    LCD_PORT &= ~(1<<LCD_RS); // RS = 0 for command
    LCD_PORT |= (1<<LCD_EN);
    _delay_ms(1);
    LCD_PORT &= ~(1<<LCD_EN);
    _delay_ms(1);
    LCD_PORT = (LCD_PORT & 0x87) | ((cmd << 4) & 0xF0); // Low nibble
    LCD_PORT |= (1<<LCD_EN);
    _delay_ms(1);
    LCD_PORT &= ~(1<<LCD_EN);
    _delay_ms(1);
}

```

```

}
void lcd_data(unsigned char data){
    LCD_PORT = (LCD_PORT & 0x87) | (data & 0xF0); // High nibble
    LCD_PORT |= (1<<LCD_RS); // RS = 1 for data
    LCD_PORT |= (1<<LCD_EN);
    _delay_ms(1);
    LCD_PORT &= ~(1<<LCD_EN);
    _delay_ms(1);
    LCD_PORT = (LCD_PORT & 0x87) | ((data << 4) & 0xF0); // Low nibble
    LCD_PORT |= (1<<LCD_EN);
    _delay_ms(1);
    LCD_PORT &= ~(1<<LCD_EN);
    _delay_ms(1);
}
void lcd_init(void){
    LCD_DDR = 0xFF; // All LCD pins as output
    _delay_ms(20);
    lcd_command(0x02); // 4-bit mode
    lcd_command(0x28); // 2 line, 5x8 font
    lcd_command(0x0C); // Display on, cursor off
    lcd_command(0x06); // Auto increment
    lcd_command(0x01); // Clear screen
    _delay_ms(2);
}
void lcd_string(const char *str){
    while(*str) lcd_data(*str++);
}
void lcd_clear(void){
    lcd_command(0x01);
    _delay_ms(2);
}
void lcd_set_cursor(uint8_t row, uint8_t col){
    uint8_t address = (row == 0) ? 0x80 : 0xC0;
    lcd_command(address + col);
}
}

```

Result:

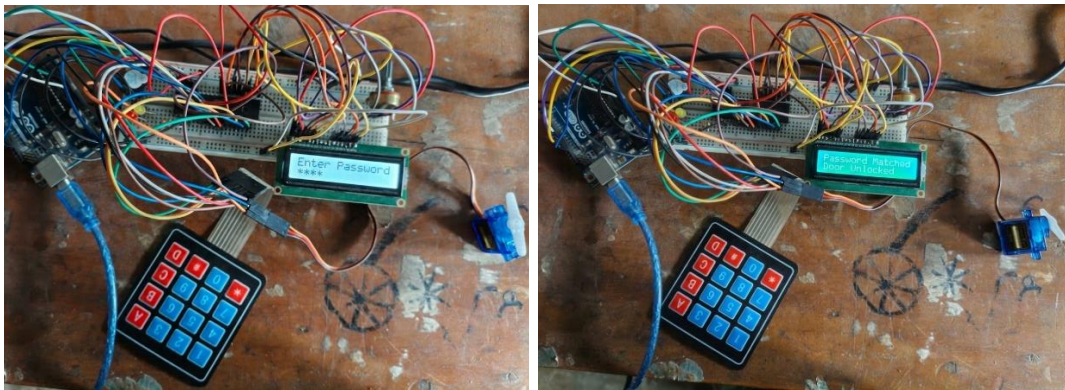


Fig 2: It successfully rotates the servo after entering correct password

Discussion:

The keypad works using a row-column scanning method. The microcontroller sets one row low at a time and reads column inputs. If a key is pressed, the corresponding column becomes low,

allowing detection of the pressed key. The LCD receives commands and data through the `lcd.h` library, simplifying control operations such as initialization, cursor positioning, and character display. Using Arduino as a burner (ISP) eliminates the need for a dedicated programmer, making the system cost-effective and flexible. Challenges faced may include: Key debounce issues, Incorrect wiring of rows and columns, Timing mismatch in LCD initialization.

The system integrates **input (keypad)**, **processing (ATmega32)**, and **output (LCD + servo motor)** into a complete embedded application. The keypad uses **row-column scanning** to detect pressed keys. The entered password is stored and compared using `strcmp()`. The LCD provides real-time feedback to the user. The servo motor is controlled using **PWM generated by Timer1** of ATmega32. If the password matches: Servo rotates (unlock position). If incorrect: System denies access and remains locked and start to sound beep. Important considerations: Proper **PWM calibration** for servo angles are really tough to set. Debouncing keypad input was control from inside and 1k ohm resistance were attached. For secure password storage I added a shift bit so normally it's unknown to others.

Conclusion:

The experiment successfully demonstrated how to interface a 4×4 keypad with ATmega32 and display inputs on an LCD. It enhanced understanding of input scanning techniques, LCD interfacing, and microcontroller programming using Arduino as ISP. The project reinforces several important concepts in embedded systems engineering, including GPIO configuration, peripheral interfacing, keypad matrix scanning, interrupt or polling-based sensor reading, software debouncing, and real-time display output using segment lookup tables. The modular firmware design allows for straightforward modification and extension of the system.

The experiment successfully demonstrates a **password-based door locking system** using ATmega32. It combines: Human interaction (keypad), Processing logic (microcontroller), Mechanical actuation (servo motor). This system is widely used in: Smart homes, Office security systems, Electronic safes. It provides a strong foundation for building advanced **IoT-based security systems** in the future.

In conclusion, this project serves as a well-rounded example of embedded system design that combines hardware interfacing, firmware logic, and practical utility. It demonstrates that even a modest 32-bit microcontroller, when properly programmed and interfaced, is capable of implementing responsive and reliable multi-input control systems.

Reference:

- [1] Atmel Corporation, "ATmega32/ATmega32L Datasheet," Atmel, San Jose, CA, Rev. 2486Z–AVR–02/2013. [Online]. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-2486-32-bit-AVR-microcontroller-ATmega32_L_datasheet.pdf
- [2] Microchip Technology Inc., "AVR Instruction Set Manual," Microchip Technology, Rev. DS40002198B, 2021. [Online]. Available: <https://www.microchip.com/en-us/application-notes/an0856>
- [3] Arduino, "ArduinoISP," Arduino Reference Documentation, 2023. [Online]. Available: <https://docs.arduino.cc/built-in-examples/arduino-isp/ArduinoISP/>
- [4] B. Huang and S. Bhatt, Practical AVR Microcontrollers: Games, Gadgets, and Home Automation with the Microcontroller Used in the Arduino. New York: Apress, 2012.